

Laporan Tugas Kecil 1 IF2211
IF2211 Strategi Algoritma

Penyelesaian Permainan *Queens* LinkedIn

Semester II tahun 2025/2026



Disusun oleh:

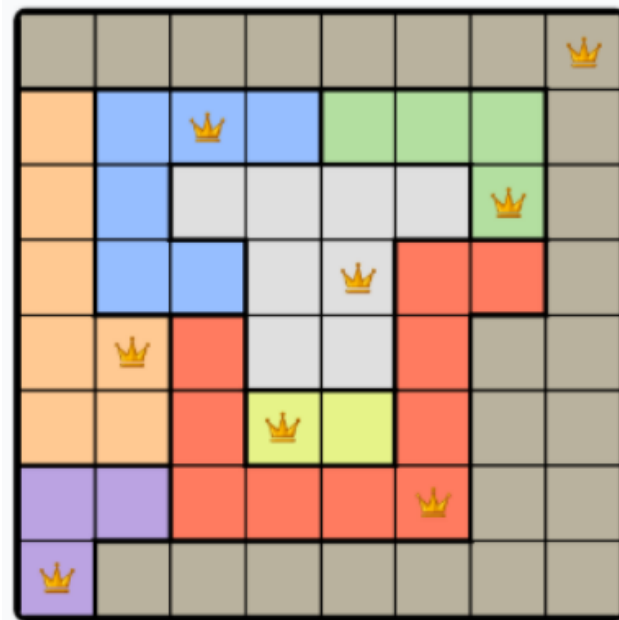
13524003 Faiq Azzam Nafidz

Daftar Isi

Pendahuluan	2
Deskripsi Persoalan	2
Algoritma yang Digunakan	3
Pengertian Algoritma <i>Brute Force</i>	3
Penentuan Algoritma	3
Analisis Kompleksitas	5
Source Program	6
Contoh Input dan Output	10
Persoalan Papan 4x4 Bersolusi	10
Persoalan Papan 5x5 Tidak Bersolusi	11
Persoalan Papan 5x5 Bersolusi	12
Persoalan Papan 6x6 Bersolusi	14
Persoalan Papan 7x7 Bersolusi	15
Penutup	17
Kesimpulan	17
Daftar Pustaka	17
Lampiran	19

Pendahuluan

Deskripsi Persoalan



Gambar 1: Tangkapan layar gim Queens

Queens adalah gim logika yang tersedia pada situs jejaring profesional LinkedIn. Tujuan dari gim ini adalah menempatkan *queen* pada sebuah papan persegi berwarna sehingga terdapat hanya satu *queen* pada tiap baris, kolom, dan daerah warna. Selain itu, satu *queen* tidak dapat ditempatkan bersebelahan dengan *queen* lainnya, termasuk secara diagonal.

Tugas anda adalah membuat program yang dapat menemukan satu solusi penempatan *queen* pada suatu papan berwarna yang diberikan, atau menampilkan bahwa tidak ada solusi yang valid. Program melakukan pencarian solusi menggunakan algoritma ***brute force***.

Algoritma yang Digunakan

Pengertian Algoritma *Brute Force*

Definisi algoritma *brute force* merupakan pendekatan lempang untuk memecahkan suatu persoalan. Definisi lain mengatakan bahwa algoritma tersebut metode yang bergantung pada kekuatan komputasi dan mencoba semua kemungkinan alih-alih menggunakan teknik lanjut untuk meningkatkan efisiensi. Sebagai contoh, persoalan yang mengharuskan seseorang menebak 4 digit kode angka yang benar terhadap suatu gembok berangka diselesaikan dengan algoritma *brute force* dengan mencoba seluruh kemungkinan dari 0000, 0001, 0002, dan seterusnya sampai 9999.

Algoritma ini didasarkan pada pernyataan persoalan serta definisi konsep yang dilibatkan untuk memecahkan persoalan tersebut. Sifat dari algoritma ini adalah sangat sederhana, langsung, dan jelas.

Penentuan Algoritma

Diketahui bahwa aturan penempatan *queen* pada papan permainan adalah sebagai berikut.

1. Hanya ada satu *queen* pada tiap baris, kolom, dan warna;
2. *Queen* tidak dapat ditempatkan bersebelahan dengan *queen* lain (termasuk secara diagonal).

Dari aturan tersebut, dapat ditentukan bahwa *queen* yang perlu ditaruh pada papan permainan sama banyaknya dengan banyak baris, banyak kolom, atau banyak warna yang ada pada papan.

Dengan algoritma *brute force*, pemikiran pertama yang muncul untuk menyelesaikan persoalan ini adalah untuk mencoba semua kemungkinan penempatan *queen* pada papan dan mengecek setiap posisi yang dicoba tersebut apakah valid dengan peraturan yang ada. Algoritma dimulai dengan menaruh setiap *queen* di baris pertama (sehingga *queen* memenuhi baris pertama, mengingat bahwa banyak *queen* sama dengan banyak kolom yang ada). Posisi ini menjadi posisi awal pengiterasian tiap kemungkinan penempatan *queen* pada papan permainan.

Dengan posisi awal ditentukan, perlu ditentukan juga bagaimana mengiterasi posisi tersebut ke posisi selanjutnya. Hal ini bisa dilakukan dengan menentukan satu *queen* yang ingin dipindahkan. Misalkan setiap petak pada papan diberi penomoran (dimulai dari 0 agar selaras dengan kode program) seperti contoh berikut.

			
4	5	6	7
8	9	10	11
12	13	14	15

Gambar 2: Contoh penomoran petak pada papan berukuran 4x4

Queen yang dipindah ditentukan sebagai *queen* yang berada pada posisi dengan angka terbesar pada papan (dalam contoh tersebut adalah *queen* di petak nomor 3), sebut saja *queen* ke- n . *Queen* ke- n akan berpindah dengan menaikkan posisinya sebanyak satu petak dengan urutan pemindahan sesuai urutan penomoran petak. Ketika *queen* tersebut tidak memiliki petak lagi untuk berpindah, jadikan *queen* sebelum *queen* tersebut, yaitu *queen* ke- $(n-1)$ menjadi *queen* yang dipindahkan (dalam contoh di atas yaitu *queen* di petak nomor 2). Ketika sudah memindahkan *queen* ke- $(n-1)$, *queen* ke- n ditempatkan kembali di posisi tepat setelah *queen* ke- n dan mulai kembali memindahkan *queen* ke- n sampai akhir. Hal ini dilakukan terus menerus sampai *queen* ke- $(n-1)$ tidak memiliki tempat berpindah ketika menjadi *queen* yang dipindahkan, menyebabkan *queen* ke- $(n-2)$ perlu dipindahkan, lalu *queen* ke- $(n-3)$ dipindahkan, dan seterusnya sampai *queen* ke-1 dipindahkan sampai posisi akhir. Dengan begitu, seluruh konfigurasi *queen* pada papan dapat teriterasi seluruhnya.

Berikut adalah jabaran algoritma di atas dalam bentuk poin-poin.

1. Tempatkan setiap *queen* pada baris pertama.
2. Pindahkan *queen* berposisi akhir dengan menaikkan posisinya sebanyak satu.
3. Letakkan *queen-queen* yang sempat dihilangkan berurutan tepat setelah *queen* yang sedang dipindahkan jika ada.
4. Cek apakah posisi *queen-queen* tersebut valid atau tidak. Jika valid, tetapkan posisi *queen-queen* tersebut sebagai solusi. Jika tidak, lanjutkan algoritma.
5. Ulangi dari langkah kedua sampai *queen* berposisi akhir tidak memiliki tempat untuk berpindah.

6. Hilangkan *queen* berposisi akhir sehingga *queen* sebelumnya menjadi *queen* berposisi akhir yang baru.
7. Ulangi dari langkah kedua sampai *queen* pertama tidak memiliki tempat untuk berpindah atau ditemukan posisi yang valid.

Analisis Kompleksitas

Algoritma tersebut bisa memecahkan persoalan dengan papan berukuran kecil. Lama penyelesaian persoalan tersebut bertambah secara eksponensial seiring membesarnya papan pada persoalan sehingga tidak disarankan untuk digunakan pada papan yang cukup besar, Hal ini disebabkan kompleksitas waktu algoritma yang terhitung cukup tinggi.

Lama waktu terburuk algoritma berjalan untuk menemukan solusi persoalan didapat ketika persoalan tidak memiliki solusi sama sekali sehingga algoritma perlu mengiterasi seluruh kemungkinan posisi *queen*. Anggap banyak *queen* dalam persoalan adalah n , maka banyak kemungkinan posisi *queen* pada papan sama saja seperti mengambil n petak pada papan berpetak $n \times n$ yang dialokasikan sebagai tempat posisi setiap *queen*. Oleh karena itu, jumlah operasi pengecekan kevalidan posisi *queen-queen* pada papan pada kemungkinan terburuk adalah

$$C(n^2, n). \quad (1)$$

Sayangnya perhitungan perhitungan kompleksitas untuk

$$C(n^2, n) \quad (2)$$

sulit atau bahkan tidak bisa dihitung tanpa menggunakan metode-metode aproksimasi bilangan faktorial. Walau begitu, dapat dilihat bahwa algoritma tersebut secara waktu sangat tidak efisien karena waktu bertambah secara dikuadratkan lalu difaktorialkan oleh fungsi kombinatorial. Belum lagi ketika menghitungnya dengan memperhatikan kompleksitas validasi posisi *queen-queen* pada papan.

Source Program

Implementasi dari algoritma yang sudah dipaparkan di bab sebelumnya diimplementasikan menggunakan bahasa Java. Berikut adalah fungsi-fungsi inti dari algoritma yang digunakan.

```
1 public class Algorithm {
2     public static boolean isDiagonalNeighbor(int posX, int
3         posY, int newX, int newY) {
4         return (posX - newX == 1 && posY - newY == 1) || (
5             posX - newX == -1 && posY - newY == 1) || (posX -
6                 newX == 1 && posY - newY == -1) || (posX - newX
7                     == -1 && posY - newY == -1);
8     }
9
10    public static boolean isValidAll(Matrix board, int[][]
11        position) {
12        boolean result = true;
13        int n = board.getRow();
14
15        for(int i = 0; i < n-1; i++) {
16            for(int j = i+1; j < n; j++) {
17                int posX1 = position[i][0];
18                int posY1 = position[i][1];
19                int posX2 = position[j][0];
20                int posY2 = position[j][1];
21                if(posY1 == -1 || posY2 == -1) break;
22                if(posX1 == posX2 || posY1 == posY2 || board
23                    .getElmt(posY1, posX1) == board.getElmt(
24                        posY2, posX2) || isDiagonalNeighbor(posX1
25                            , posY1, posX2, posY2)) {
26                    result = false;
27                    break;
28                }
29            }
30        }
31
32        return result;
33    }
34 }
35
36 public class Controller implements Initializable {
37     void exhaustiveSolveGUI(Matrix board, Result result) {
38         boolean showStep = checkBoxShowStep.isSelected();
39         int nStep;
40         if(showStep) nStep = Integer.parseInt(textNStep.
41             getText());
42         else nStep = 1;
43         int mStep;
```

```

35         if(showStep) mStep = Integer.parseInt(textMStep.
36             getText());
37     else mStep = 1;
38
39     backgroundTask = new Task<Void>() {
40         @Override
41         public Void call() {
42             long startTime = System.nanoTime();
43             int cases = 1;
44
45             int[][] position = new int[n][2];
46             for(int i = 0; i < n; i++) {
47                 position[i][0] = i;
48                 position[i][1] = 0;
49             }
50
51             int placed = n;
52             int toMove = n-1;
53             boolean possible = true;
54
55             while(toMove < n && possible) {
56                 if (isCancelled()) {
57                     System.out.println("Task cancelled")
58                     ;
59                     return null;
60                 }
61
62                 int[][] snapshot = new int[n][2];
63                 for(int i = 0; i < n; i++) {
64                     snapshot[i][0] = position[i][0];
65                     snapshot[i][1] = position[i][1];
66                 }
67                 final int[][] finalPos = snapshot;
68                 final int finalPlaced = placed;
69                 if(!isCancelled() && showStep && cases %
70                     nStep == 0){
71                     Platform.runLater(() -> {
72                         if (backgroundTask == null ||
73                             backgroundTask.isCancelled())
74                             {
75                                 return;
76                             }
77                         if(finalPlaced == n){
78                             displayQueen(finalPos);
79                         }
80                     });
81                 }
82                 try {
83                     Thread.sleep(mStep);
84                 } catch (InterruptedException e) {

```



```

79         Thread.currentThread().interrupt
80         ();
81     }
82 }
83
84     if(placed == n) {
85         if(Algorithm.isValidAll(board,
86             position)) {
87             break;
88         }
89         cases++;
90     }
91
92     int newX = position[toMove][0];
93     int newY = position[toMove][1];
94
95     if(newX == -1) {
96         newX = position[toMove-1][0];
97         newY = position[toMove-1][1];
98         placed++;
99     }
100
101     newX++;
102     if(newX >= n) {
103         newX = 0;
104         newY += 1;
105     }
106     if(newY >= n) {
107         position[toMove][0] = -1;
108         position[toMove][1] = 0;
109         placed--;
110         toMove--;
111         if(toMove < 0) {
112             possible = false;
113             break;
114         }
115         continue;
116     }
117     else {
118         position[toMove][0] = newX;
119         position[toMove][1] = newY;
120     }
121
122     if(toMove < n-1) {
123         toMove++;
124     }
125 }
126
127 long endTime = System.nanoTime();

```

```

126         long time = endTime - startTime;
127
128         result.position = position;
129         result.calculationTime = time;
130         result.cases = cases;
131
132         return null;
133     }
134 };
135 backgroundTask.setOnSucceeded(e -> {
136     displayResult(result);
137 });
138 backgroundTask.setOnCancelled(e -> {
139     for(int row = 0; row < n; row++) {
140         for(int col = 0; col < n; col++) {
141             colorGrid[row][col].setGraphic(null);
142         }
143     }
144     textStatusRight.setText("Calculation cancelled")
145     ;
146 });
147 new Thread(backgroundTask).start();
148 }

```

Contoh Input dan Output

Persoalan Papan 4x4 Bersolusi

Contoh penyelesaian persoalan papan 4x4 yang memiliki solusi.

Input:

CABB

CABB

CAAA

AADD



Gambar 3: Input Test 1

Output:

C#BB

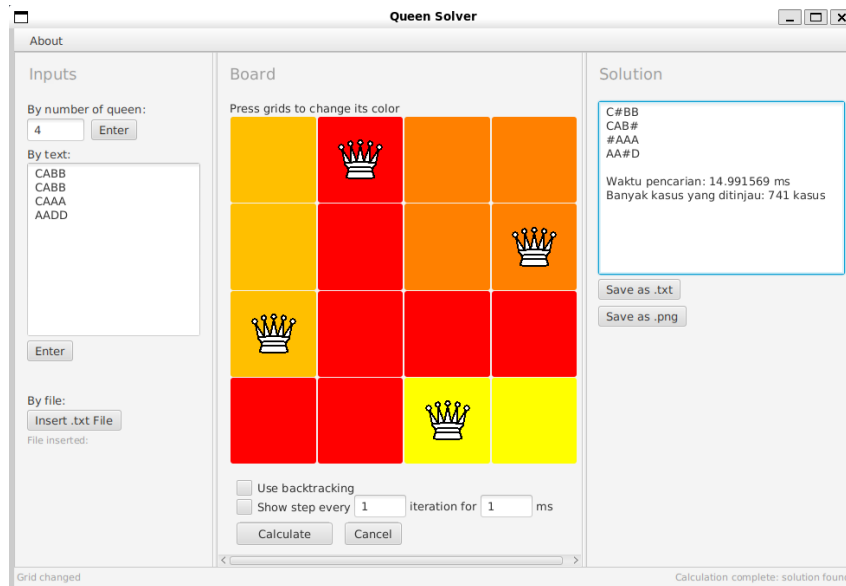
CAB#

#AAA

AA#D

Waktu pencarian: 14.991569 ms

Banyak kasus yang ditinjau: 741 kasus



Gambar 4: Output Test 1

Persoalan Papan 5x5 Tidak Bersolusi

Contoh penyelesaian persoalan papan 5x5 yang tidak memiliki solusi.

Input:

AAAAB

ACCDD

ACCCD

AAADD

AAAEE



Gambar 5: Input Test 2

Output:

Tidak ada solusi!

Waktu pencarian: 35.025546 ms

Banyak kasus yang ditinjau: 53131 kasus



Gambar 6: Output Test 2

Persoalan Papan 5x5 Bersolusi

Contoh penyelesaian persoalan papan 5x5 yang memiliki solusi.

Input:

AAABB

ACCCD

CCCCD

CCEDD

EEEE



Gambar 7: Input Test 3

Output:
 AAA#B
 #CCDD
 CC#CD
 CCED#
 E#EEE

Waktu pencarian: 32.502041 ms
 Banyak kasus yang ditinjau: 28672 kasus



Gambar 8: Output Test 3

Persoalan Papan 6x6 Bersolusi

Contoh penyelesaian persoalan papan 6x6 yang memiliki solusi.

Input:

ABCDEE
AACDEE
ACCDDE
AACCDF
FFCFDF
FFFFFF



Gambar 9: Input Test 4

Output:

A#CDEE
AAC#EE
ACCD#
#ACCDF
FF#FDF
FFFF#F

Waktu pencarian: 155.356381 ms

Banyak kasus yang ditinjau: 533331 kasus



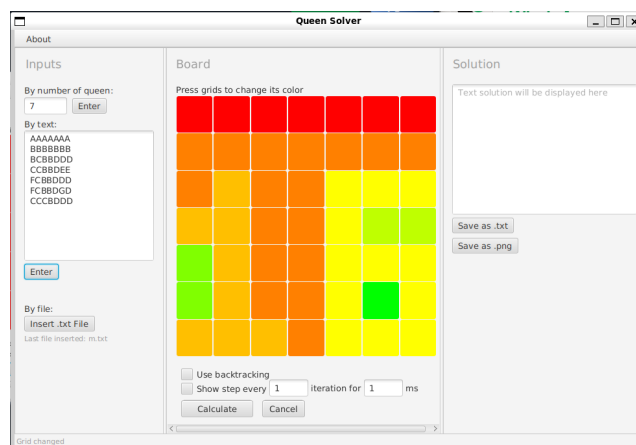
Gambar 10: Output Test 4

Persoalan Papan 7x7 Bersolusi

Contoh penyelesaian persoalan papan 7x7 yang memiliki solusi.

Input:

AAAAAAA
BBBBBBB
BCBBDDD
CCBBDEE
FCBBDDD
FCBBDDG
CCCBDDD



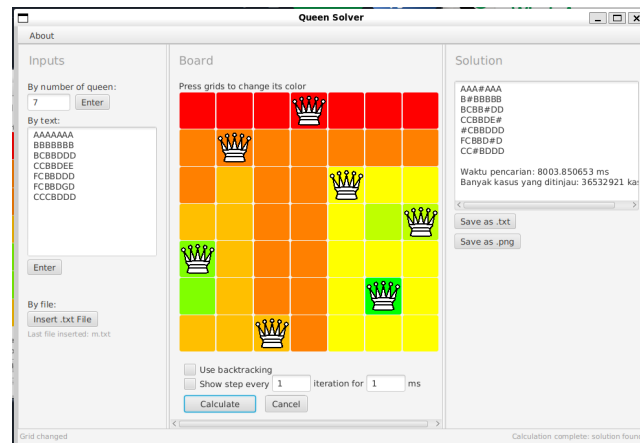
Gambar 11: Input Test 5

Output:

AAA#AAA

B#BBBBB
 BCBB#DD
 CCBBDE#
 #CBBDDD
 FCBBD#D
 CC#BDDD

Waktu pencarian: 8003.850653 ms
 Banyak kasus yang ditinjau: 36532921 kasus



Gambar 12: Output Test 5

Penutup

Kesimpulan

Penggunaan algoritma *brute force* terhadap gim *Queens* bisa dilakukan. Namun, karena algoritma *brute force* yang bersifat sederhana, penyelesaian persoalan pada gim tersebut dengan algoritma *brute force* hanya bisa digunakan pada papan berukuran kecil. Waktu penyelesaian persoalan menggunakan algoritma tersebut akan bertambah dengan sangat cepat sehingga tidak efisien untuk menyelesaikan persoalan dengan papan berukuran besar.

Daftar Pustaka

- [1] freeCodeCamp, "Brute Force Algorithms Explained," 6 Januari 2020. Tersedia di: <https://www.freecodecamp.org/news/brute-force-algorithms-explained/>. (Diakses pada 17 Februari 2026).
- [2] Rinaldi, "Algoritma Brute Force (Bagian 1)," Sekolah Teknik Elektro dan Informatika, Institut Teknologi Bandung, 2026. Tersedia di: [https://informatika.stei.itb.ac.id/~rinaldi.munir/Stmik/2025-2026/02-Algoritma-Brute-Force-\(2026\)-Bag1.pdf](https://informatika.stei.itb.ac.id/~rinaldi.munir/Stmik/2025-2026/02-Algoritma-Brute-Force-(2026)-Bag1.pdf). (Diakses pada 17 Februari 2026).

Lampiran

Tautan Repository Github: https://github.com/Shui7/Tucil1_13524003

Tugas ini disusun sepenuhnya tanpa bantuan kecerdasan buatan (Generative AI), melainkan hasil pemikiran dan analisis mandiri.

A handwritten signature in black ink, appearing to read 'Faiq Azzam Nafidz', with a stylized, cursive script.

Faiq Azzam Nafidz